

Easy Performant Outline 2D | 3D (URP/HDRP and Built-in Renderer) v3.5.9

Easy Performant Outline is a powerful tool designed to easily add customizable 2D and 3D outlines to objects in Unity. With support for various renderers and full optimization, this asset allows you to create high-quality outlines without compromising performance. Whether you are working on a 2D or 3D project, this asset offers flexible options for color, thickness, style, and more, enabling you to enhance the visual appeal and clarity of your objects in games and applications.

Table of Contents

1. [Introduction](#)
2. [Installation](#)
 - [Importing the Asset into the Project](#)
 - [Setup for Different Renderers](#)
 - [Setup for Built-in Renderer](#)
 - [Setup for URP Renderer](#)
 - [Setup for HDRP Renderer](#)
 - [Troubleshooting Installation Issues](#)
3. [Core Components](#)
 - [Outliner settings](#)
 - [Outlinable settings](#)
4. [Advanced Settings](#)
 - [Masking](#)
 - [2D Functionality](#)
5. [Integration](#)
 - [Working with the C# API](#)
 - [DOTween Support](#)
6. [Frequently Asked Questions \(FAQs\)](#)
7. [Contact Information](#)

If you have any questions about EP Outline, please check the [FAQs](#) first, as you might find your answer there. If you're unable to find a solution, feel free to report the issue or reach out to us via email at pirate.parrot.software@gmail.com or through our [Discord](#) community.

When reporting a bug, it helps us a lot if you include clear steps to reproduce the issue. This allows us to identify and fix the problem faster.

Please bear with us if it takes some time to respond — we receive many messages daily, but we'll get back to you as soon as possible.

Lastly, if you have a feature idea or improvement suggestion, don't hesitate to share it with us. Some of our best features have come directly from user feedback!

★ We'd Love to Hear Your Feedback! ★

If you enjoy using Easy Performant Outline, please consider leaving a review on the [Unity Asset Store](#)! Your ratings and feedback help us improve the asset and continue to develop new features. Thank you for your support! 🙏

Introduction

Easy Performant Outline is a tool for creating high-quality outlines for 2D and 3D objects in Unity. The asset allows you to easily add outlines to objects, helping to highlight them on the scene, improve visual perception, and increase interactivity. It solves the performance problem by enabling the creation of outlines with minimal resource consumption, making it an ideal choice for projects with high performance demands.

Key features:

- Easy to use: setting up outlines takes just a few minutes.
 - Supports various Unity renderers (Build-in, URP, HDRP).
 - High performance, even when using a large number of outlines.
 - Full customization of outline parameters: color, thickness, blur, and more.
 - Support for animations with DoTween for dynamic effects.
 - Compatibility with both 2D and 3D objects, including sprite support.
 - Render graph support for modern projects.
-

Requirements:

- **Supported Unity versions:** 2021.3.0 or higher.
 - **Supported platforms:** Android, iOS, PC, Mac, PS4, Oculus/Vive, WebGL and others.
 - **Supported renderers:** Standart, Universal Render Pipeline (URP), High Definition Render Pipeline (HDRP).
 - **Hardware requirements:** The asset is optimized for use with modern GPUs, but at least a mid-range GPU is recommended for optimal performance in more complex scenes.
-

Links to tutorials and promotional materials:

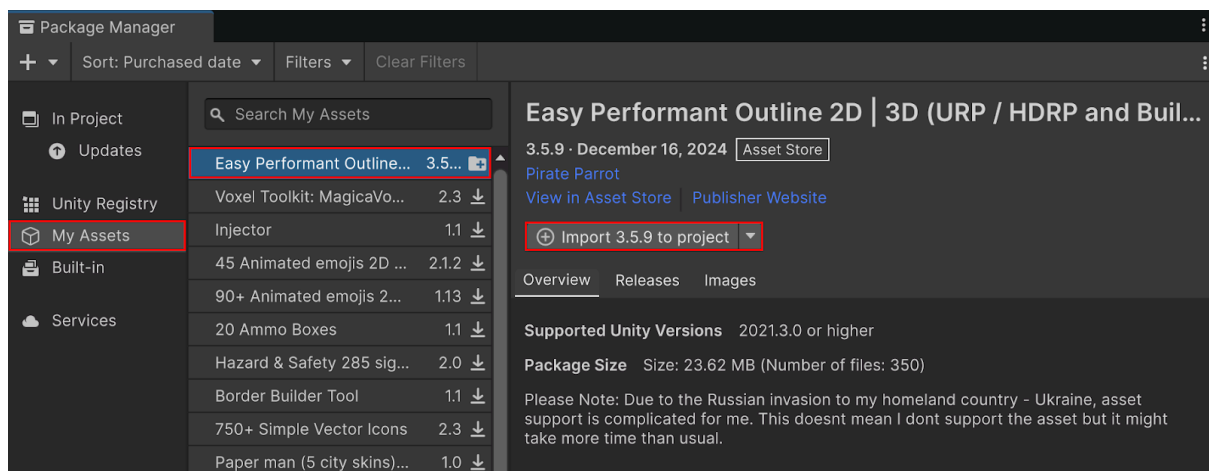
- [Full Overview of Easy Performant Outline](#) (Installation and Usage)
- [Feature Demonstration of the Asset in Action](#) (Older Video)
- [Configuring Easy Performant Outline through C#](#) (Older Video)

Installation

Importing the Asset into the Project

To install **Easy Performant Outline** using Unity Package Manager, follow these steps:

1. Open your Unity project.
2. Go to **Window > Package Manager**.
3. Go to **My Assets**, locate **Easy Performant Outline**, and click **Import**.
4. Follow the on-screen instructions to complete the import.



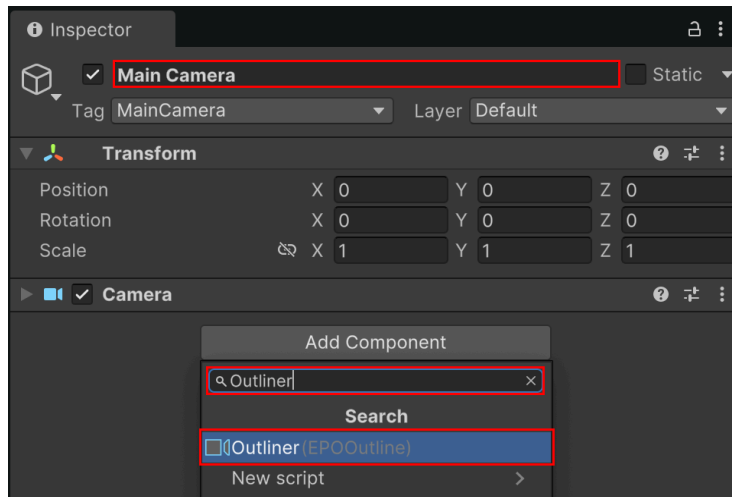
Setup for Different Renderers

Setup for Built-in Renderer

To configure **Easy Performant Outline** for the Built-in Renderer, follow these steps:

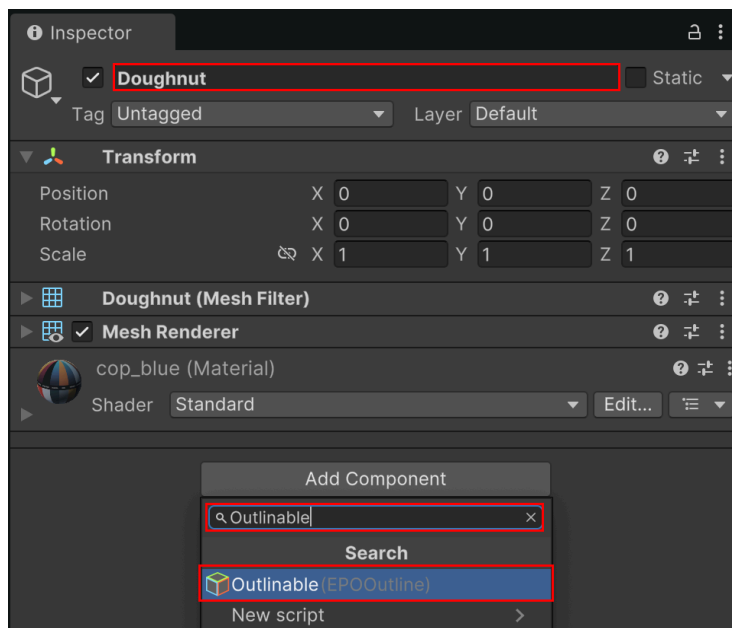
1. Add the Outliner Component

Attach the **Outliner** component to the scene camera where you want the outlines to be rendered.



2. Add the Outlinable Component

Attach the **Outlinable** component to the object(s) you want to highlight with an outline.



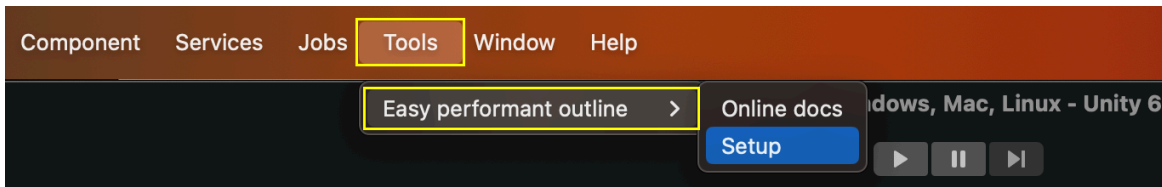
These simple steps will enable outline rendering in your scene using the Built-in Renderer.

Setup for URP Renderer

To configure Easy Performant Outline for the Universal Render Pipeline (URP), follow these steps:

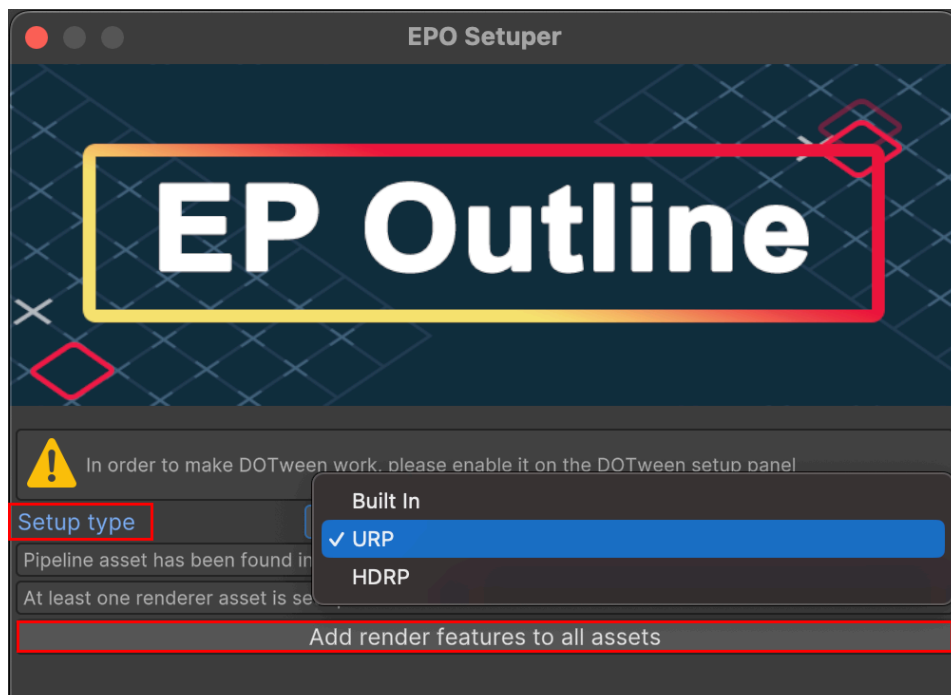
1. **Open the Setup Tool**

Go to **Tools > Easy Performant Outline > Setup** in the Unity menu bar to open the EPO Setuper.



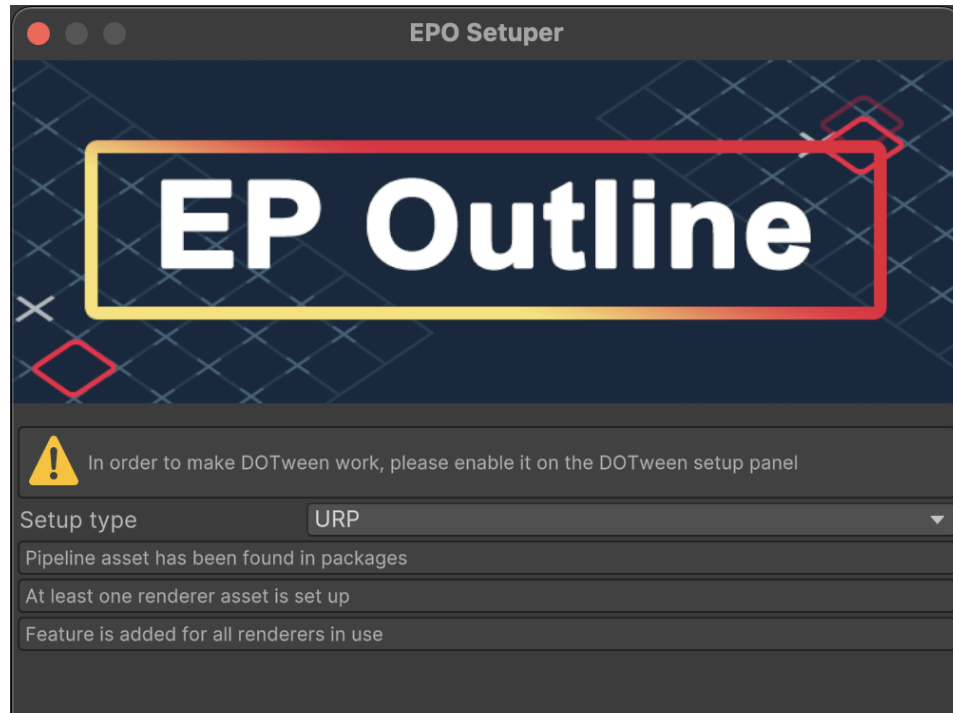
2. **Select URP and Apply the URP Outline Feature**

In the setup window, select **URP** as the render pipeline type, then click the **Add render features to all assets** button. This will automatically configure the URP Outline Feature for your project.



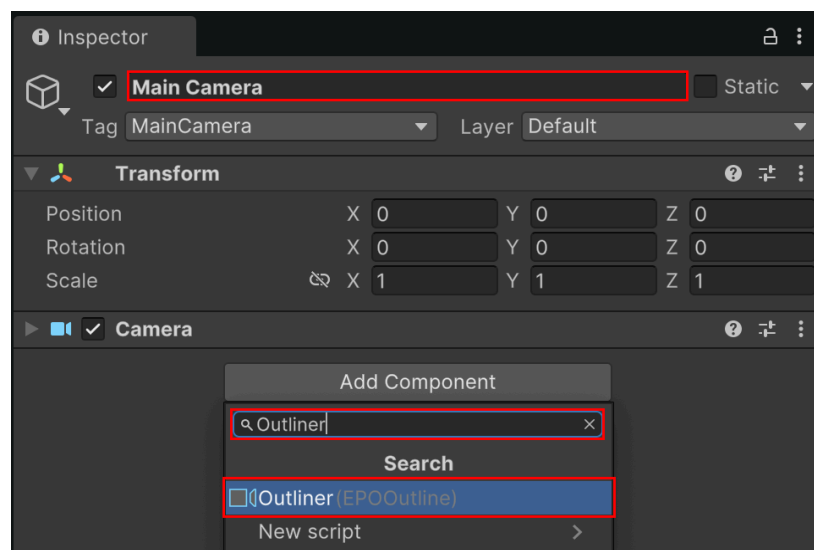
3. Verify Setup

Confirm that the URP Outline Feature is successfully added. The setup tool will notify you if the pipeline asset is detected and properly configured.



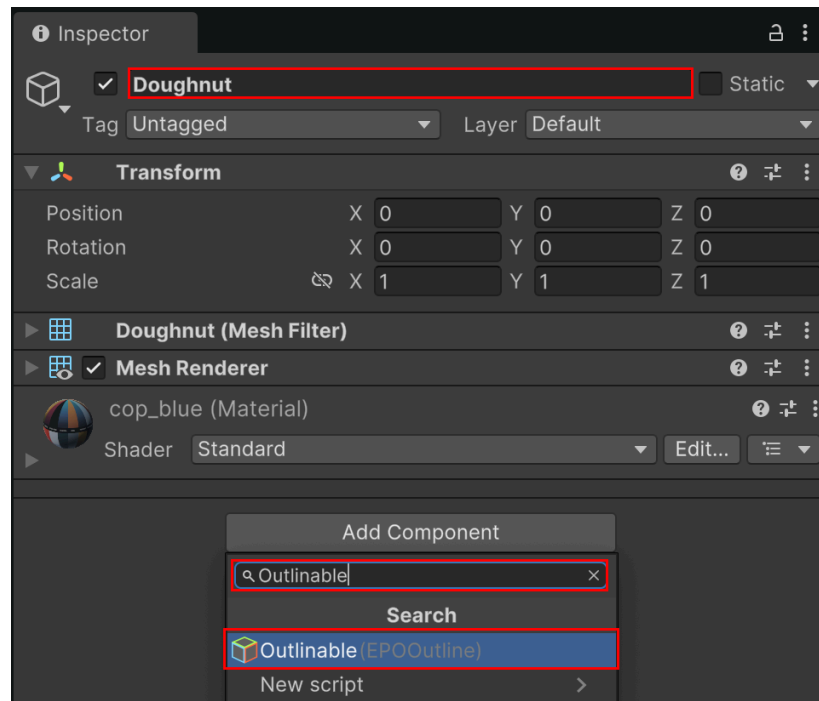
4. Add the Outliner Component

Attach the **Outliner** component to the scene camera where you want the outlines to be rendered.



5. Add the Outlinable Component

Attach the **Outlinable** component to the object(s) you want to highlight with an outline.



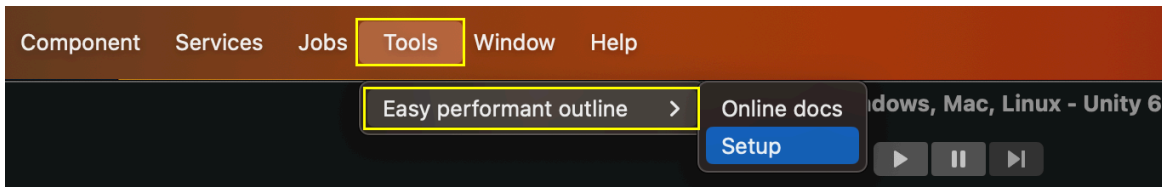
These simple steps will enable outline rendering in your scene using the URP Renderer.

Setup for HDRP Renderer

To configure Easy Performant Outline for the High Definition Render Pipeline (HDRP), follow these steps:

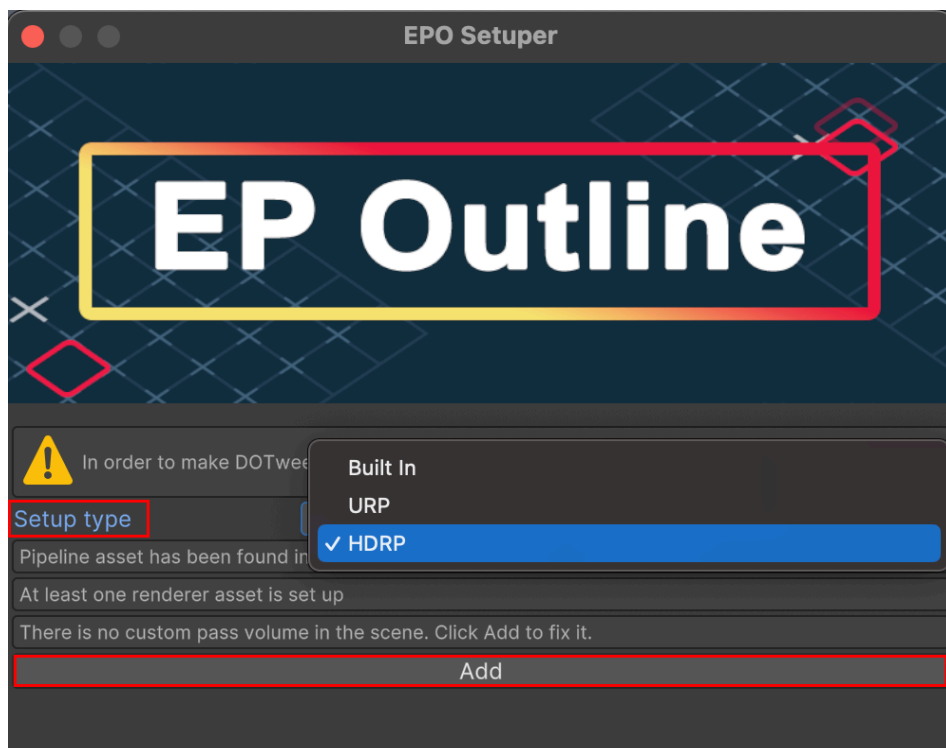
1. **Open the Setup Tool**

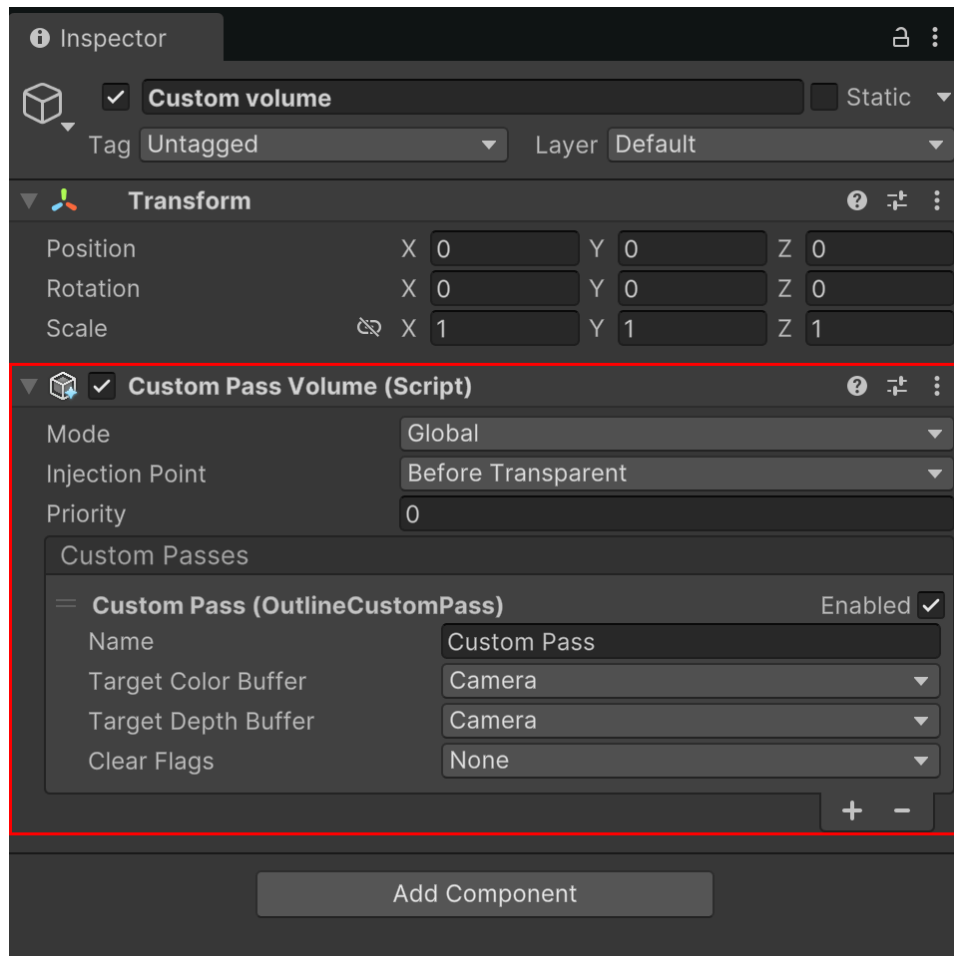
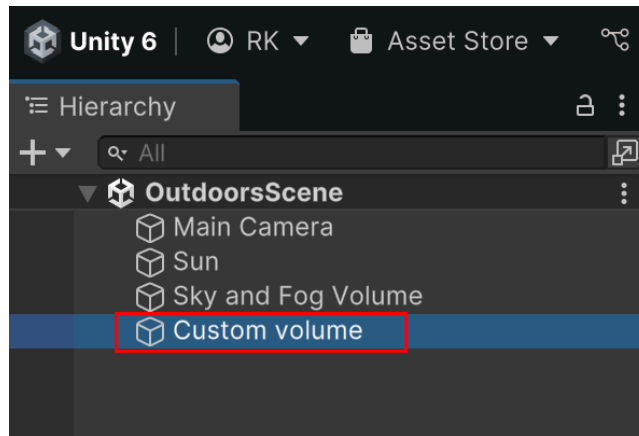
Go to **Tools > Easy Performant Outline > Setup** in the Unity menu bar to open the EPO Setuper.



2. **Select HDRP and create Custom Pass**

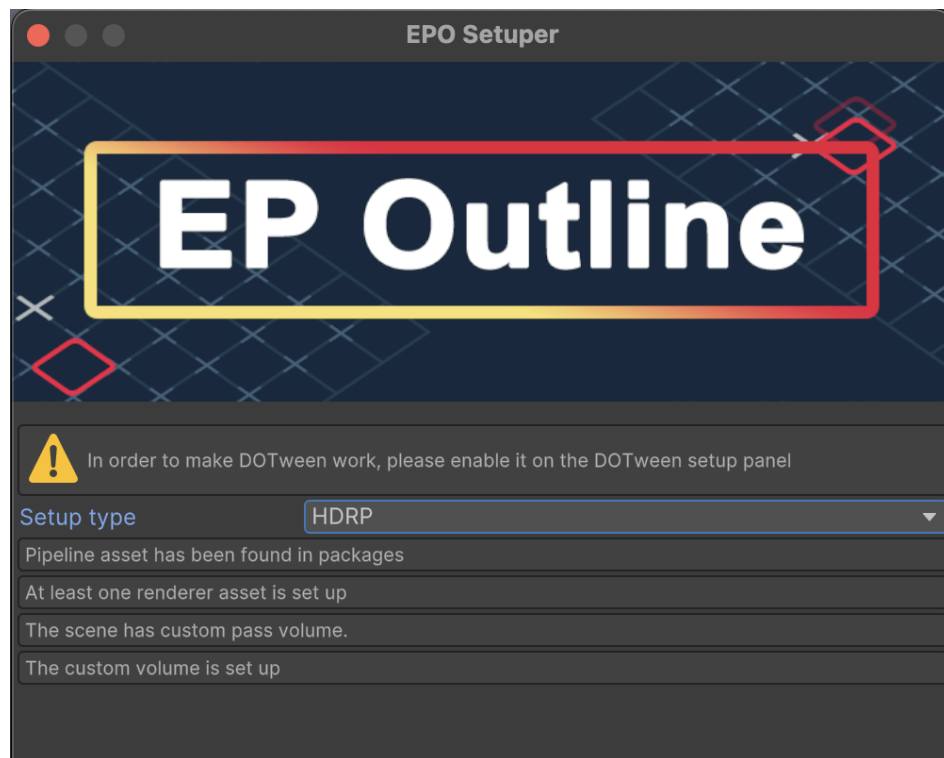
In the setup window, choose HDRP as the render pipeline type. Click the **Add** button to create a Custom Pass Volume in the scene. Next, you will be prompted to add a custom pass to the newly created volume — simply click the **Add** button again to complete the setup.





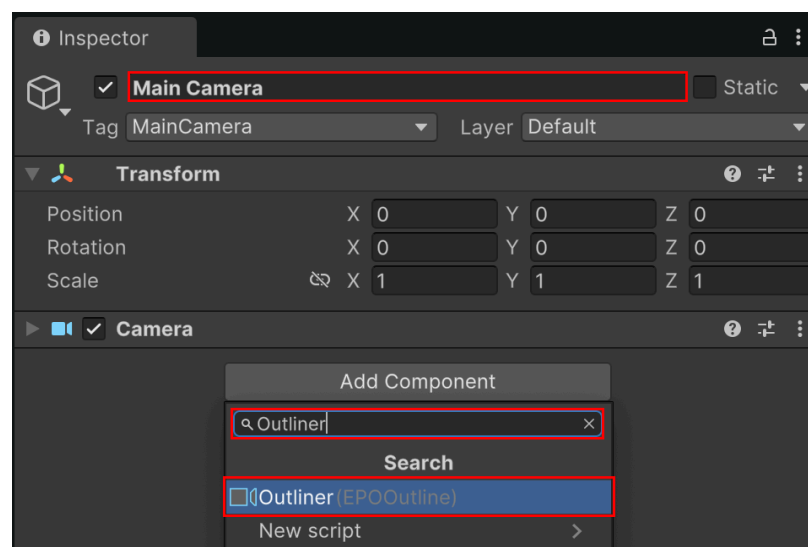
3. Verify Setup

Confirm that the HDRP Outline Feature is successfully added. The setup tool will notify you if the pipeline asset is detected and properly configured.



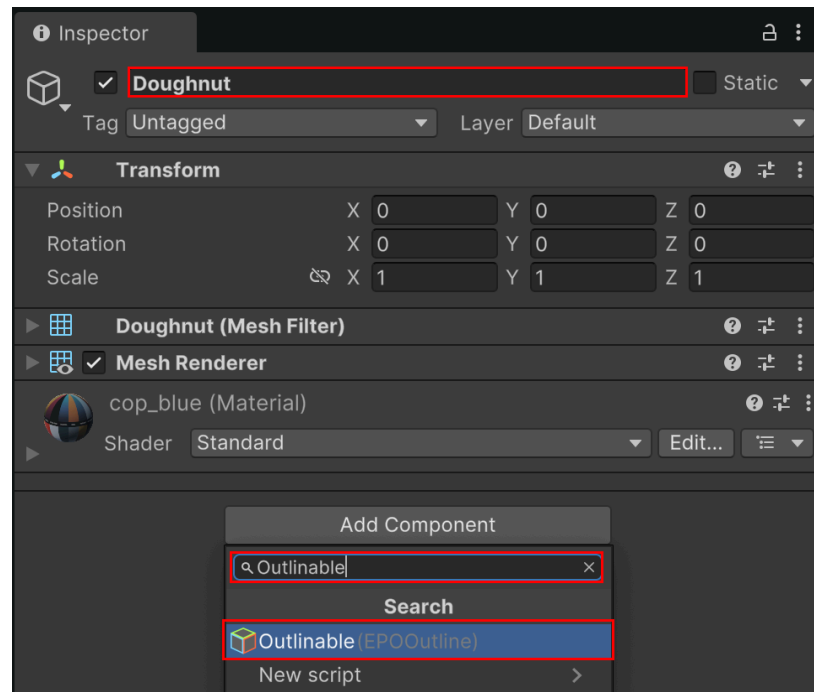
4. Add the Outliner Component

Attach the **Outliner** component to the scene camera where you want the outlines to be rendered.



5. Add the Outlinable Component

Attach the **Outlinable** component to the object(s) you want to highlight with an outline.



These simple steps will enable outline rendering in your scene using the HDRP Renderer.

Troubleshooting Installation Issues

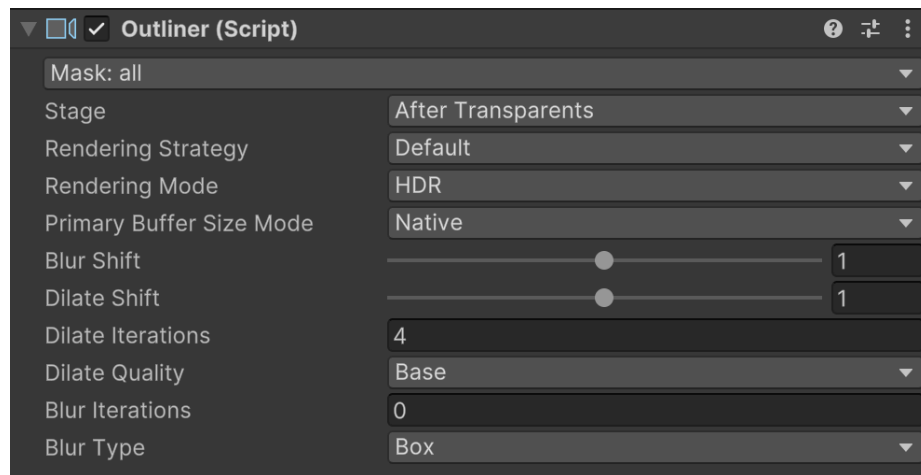
If you encounter issues during installation, here are a few things to try:

- **Unity Restart:** After importing the asset, restart Unity to ensure everything loads correctly.
- **Re-import:** If something seems missing or not working, try re-importing the package.
- **Package Compatibility:** Ensure that you are using a Unity version compatible with **Easy Performant Outline** (Unity 2021.3.0 or later).
- **Renderer Compatibility:** Make sure you've selected the correct renderer (Built-in, URP, or HDRP) in your project settings.
- **Check Console:** Open the **Console** window to check for any errors or warnings after installation. It can help identify potential setup issues.

If you still face issues, feel free to reach out via email or Discord support.

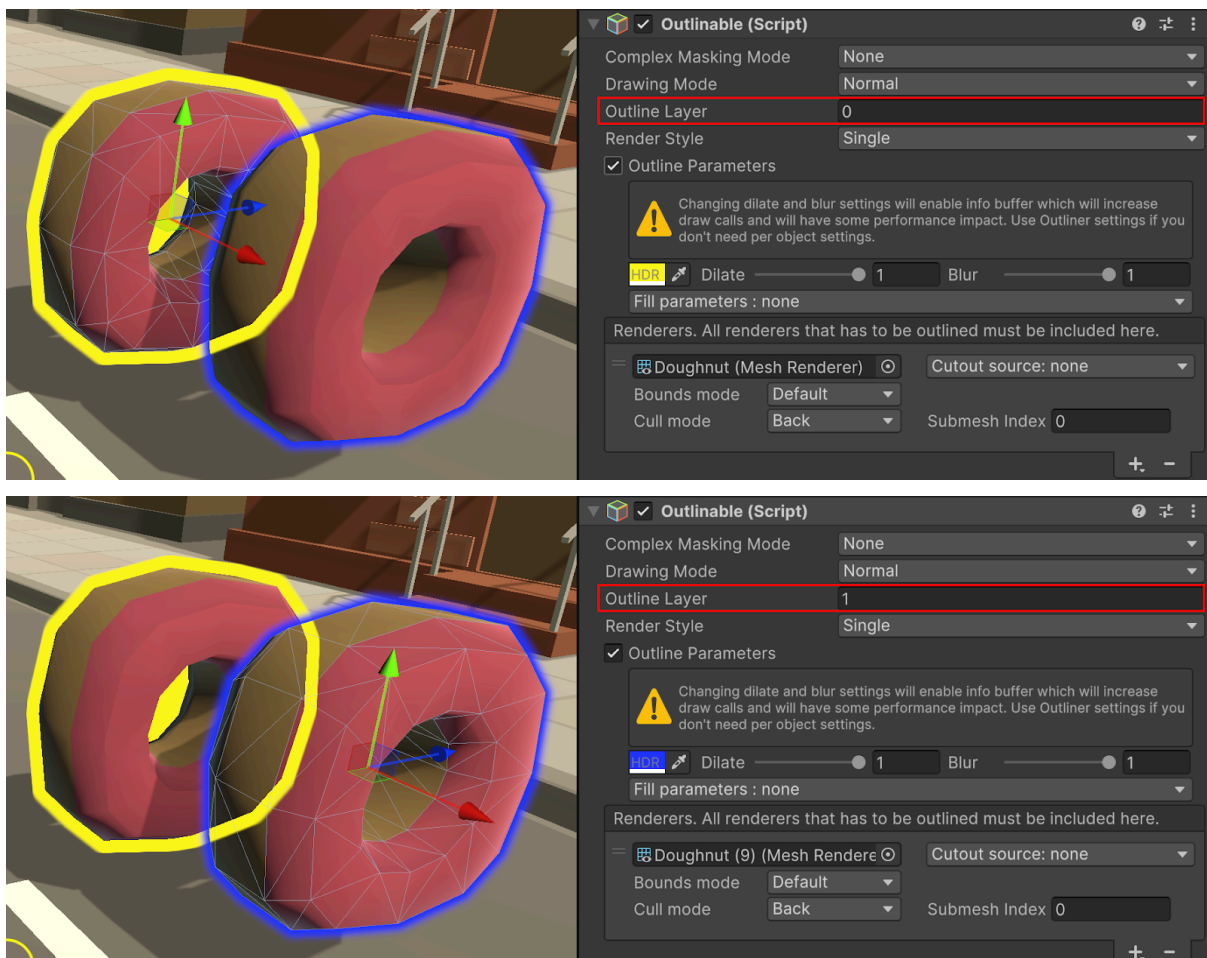
Core Components

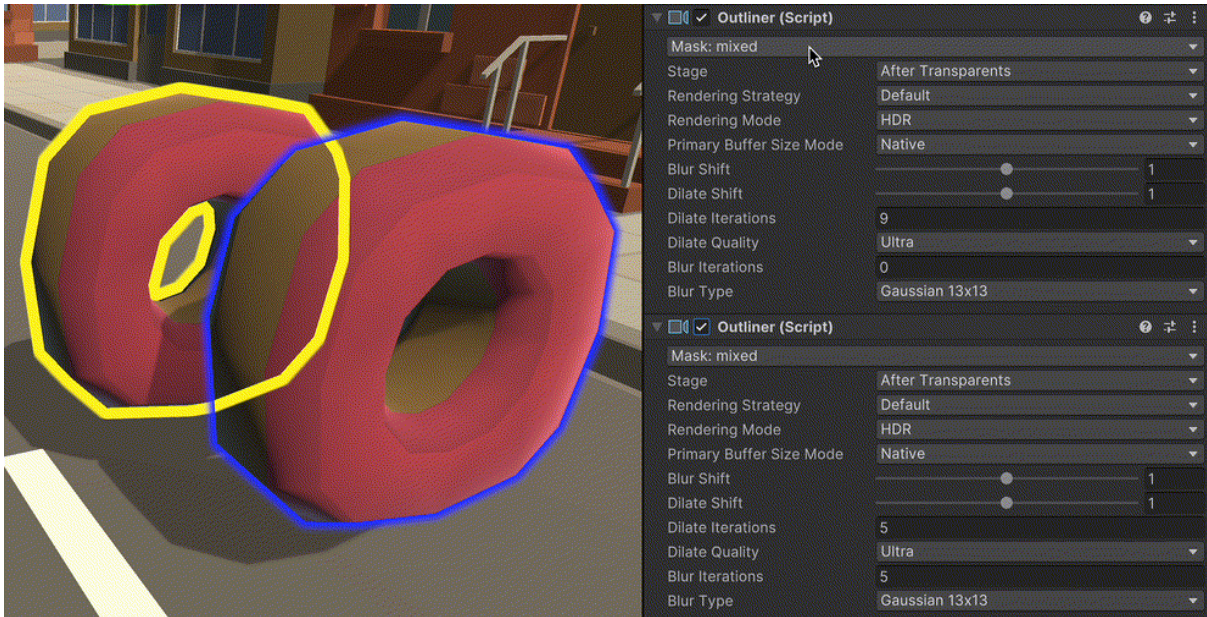
Outliner settings



Mask

Defines which layers the outliner should render. If you selected specific layers on the outlinables, you can exclude them from rendering by this outliner. You can add multiple outliners to the same camera with different layers.





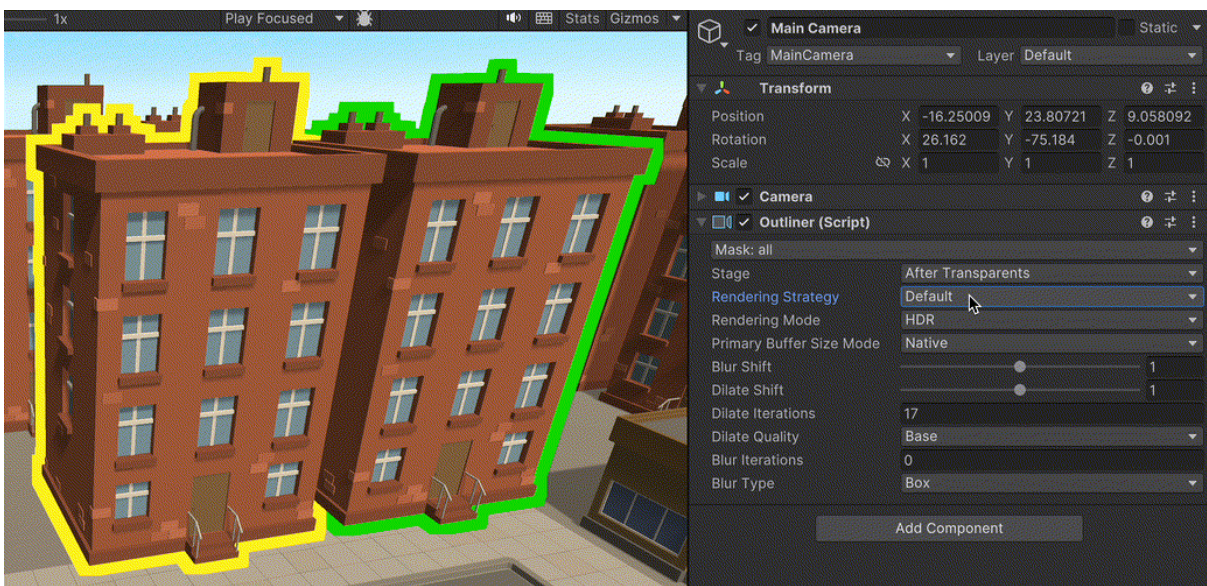
Stage

Defines the render stage for the outline.

- **Before Transparents**
The outline will render before the transparent objects are drawn.
- **After Transparents**
The outline will render after the transparent objects are drawn.

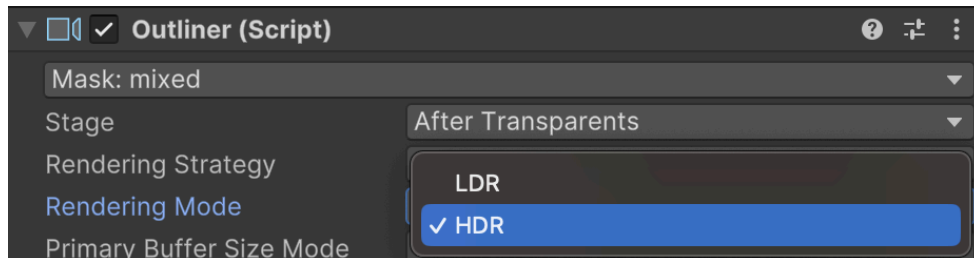
Rendering Strategy

Defines the method used for rendering the outline. For per-object outlines, the edges of outlines do not blend together. Please note that this method has some performance impact.



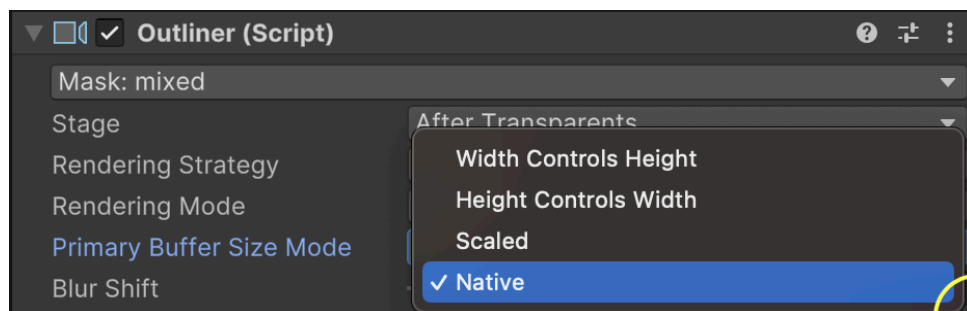
Rendering Mode

Defines whether the outliner should use HDR or LDR for the outline. (In the case of post-processing effects like bloom, HDR is important; in other cases, it might be a waste of resources.) Please note that HDR will not be used if the camera does not support HDR or if HDR is disabled in the system.



Primary Buffer Size Mode

The **Primary Buffer Size Mode** parameter allows you to choose how the outline rendering resolution is handled.

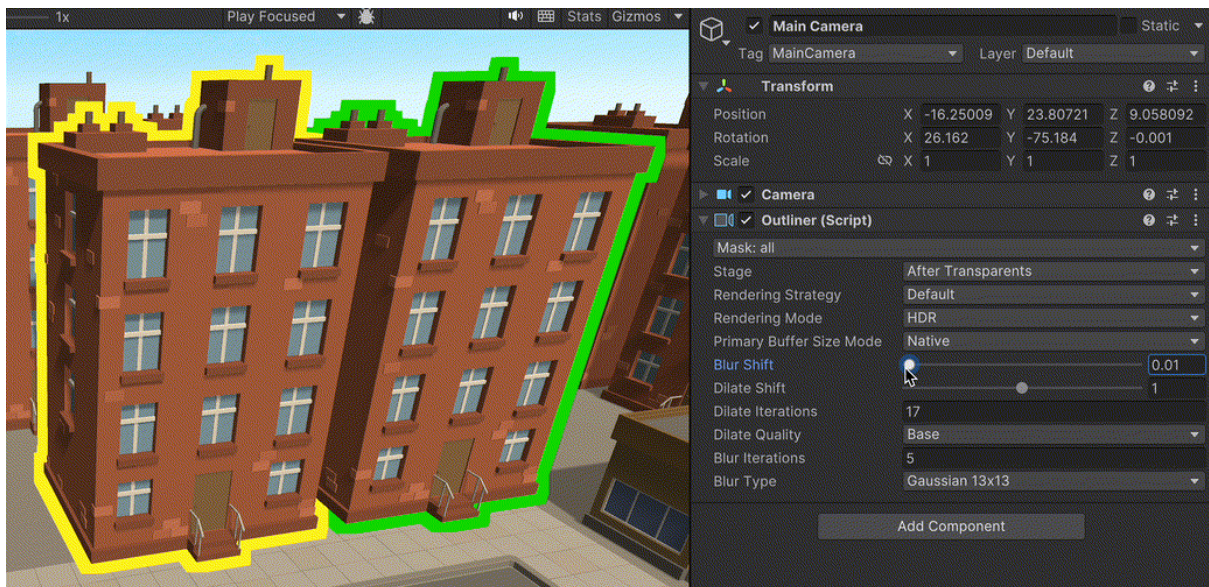


The available options include:

- **Native**
The outline is rendered at the screen's native resolution, ensuring the highest quality.
- **Scaled**
The outline resolution is scaled on both axes according to the **Primary Render Scale**.
For example, if the screen resolution is 800x600 and the **Primary Render Scale** is set to 0.5, the outline will be rendered at 400x300 resolution.
It is recommended to set this value as low as possible while maintaining the desired visual quality, as lower values improve performance.
- **Width Controls Height**
The outline's width is set based on the **Primary Size Reference**, and the height is adjusted automatically to maintain the correct aspect ratio.
- **Height Controls Width**
Similar to the previous mode, but in this case, the height is set based on the **Primary Size Reference**, and the width is scaled proportionally.

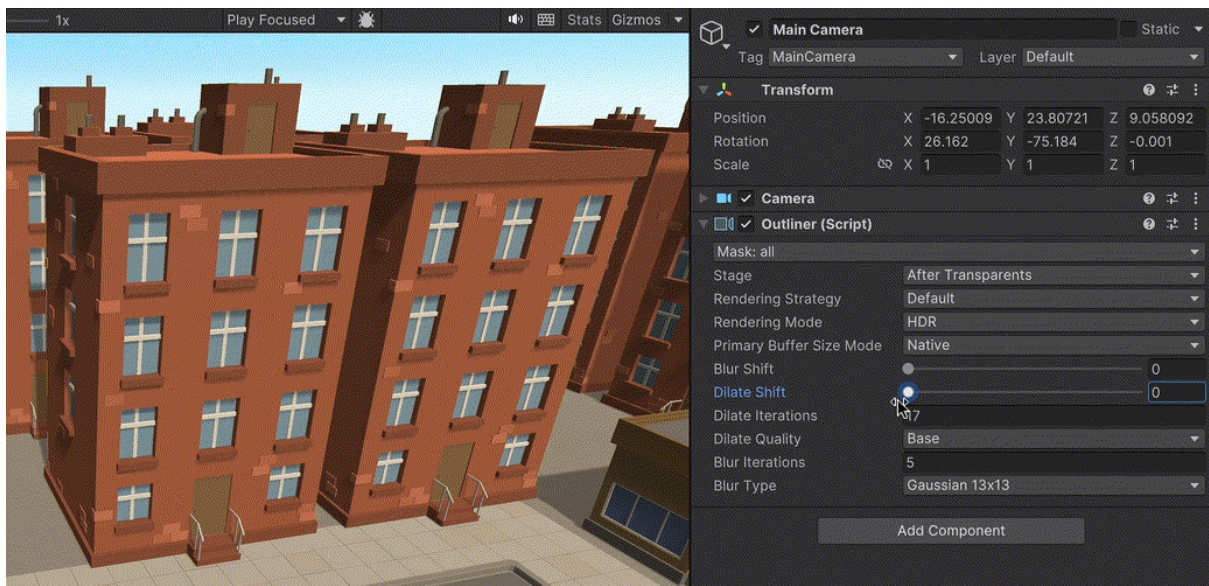
Blur Shift

Controls the amount of blur applied to the final image. Note that values greater than 1 may produce artifacts.



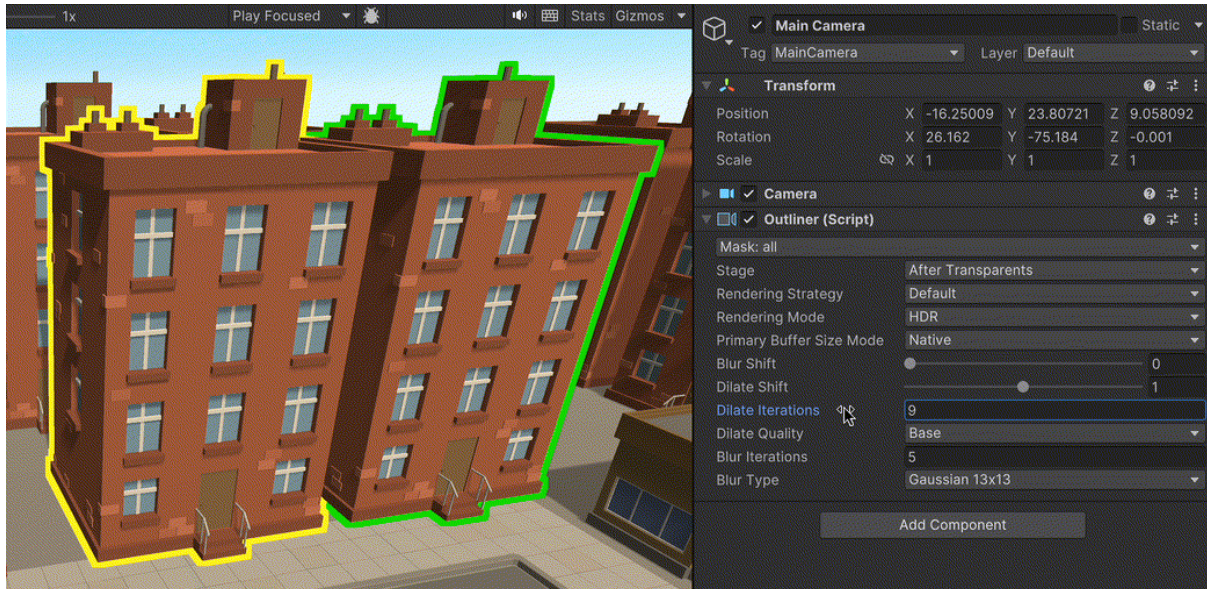
Dilate Shift

Controls the amount of dilation applied to the final image. Note that values greater than 1 may produce artifacts.



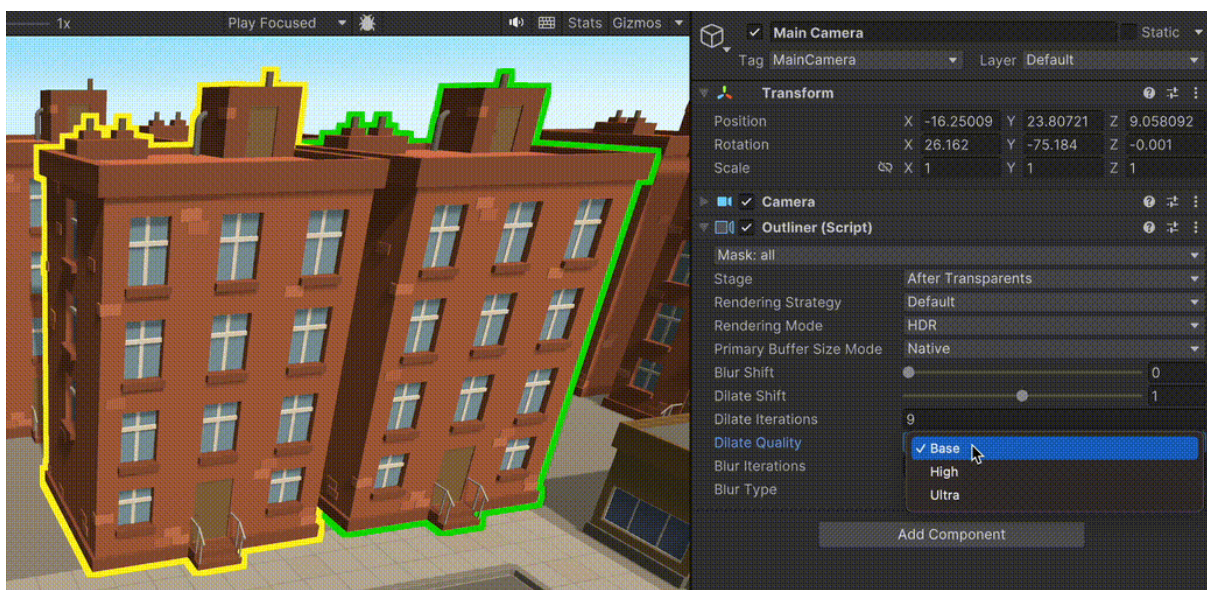
Dilate Iterations

Defines how many dilation effect iterations should be applied. A smaller value improves performance, while a larger value results in a stronger dilation effect. The difference between **shift** and **iterations** is the same as for **Blur Shift** and **Blur Iterations**.



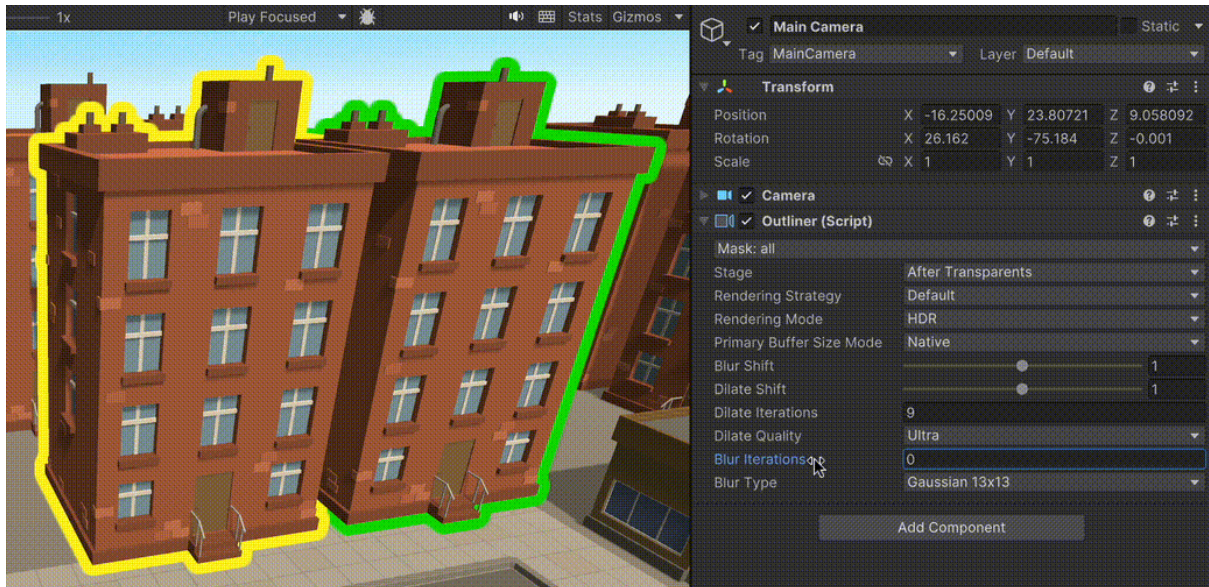
Dilate Quality

Controls the quality of the dilation effect. The **Base** setting is the most performant. **High** and **Ultra** provide better quality but are more resource-intensive. The **Base** variation dilates the outline along both axes, providing a performance-friendly yet approximate effect. The other two variations use circular sampling, which is more resource-intensive but delivers more precise results. For optimal performance, it is recommended to use the **Base** variation whenever possible.



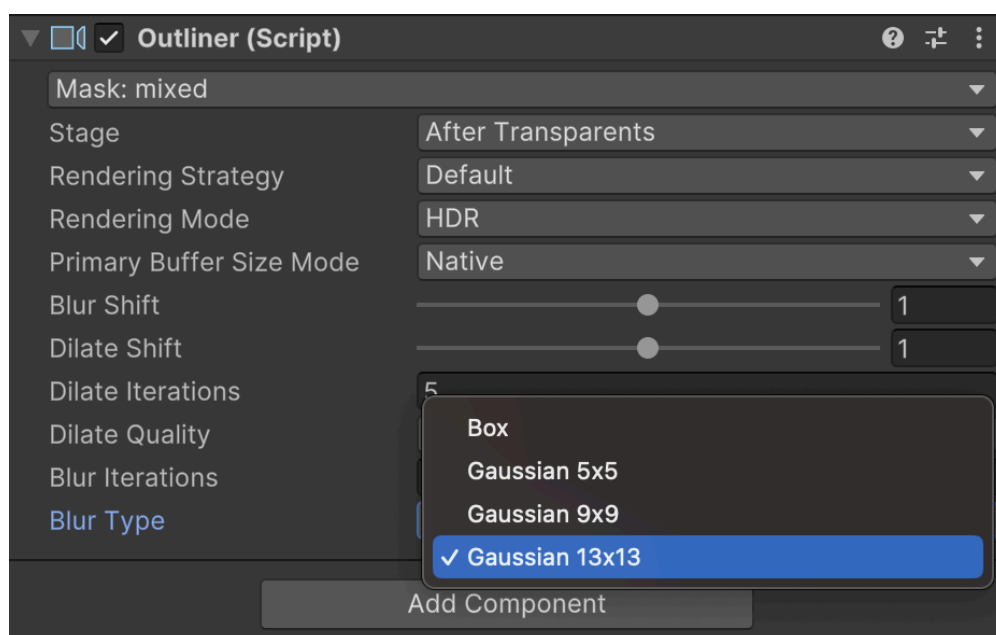
Blur Iterations

Defines how many blur effect iterations should be applied. A smaller value improves performance, while a larger value results in a stronger blur effect. The difference between **shift** and **iterations** lies in their function: **iterations** define the number of post-processing passes required to achieve the desired effect, while **shift** adjusts the intensity of the effect within its range. For better performance, prioritize using **shift** parameters to achieve the desired result and increase the **iterations** count only when absolutely necessary.

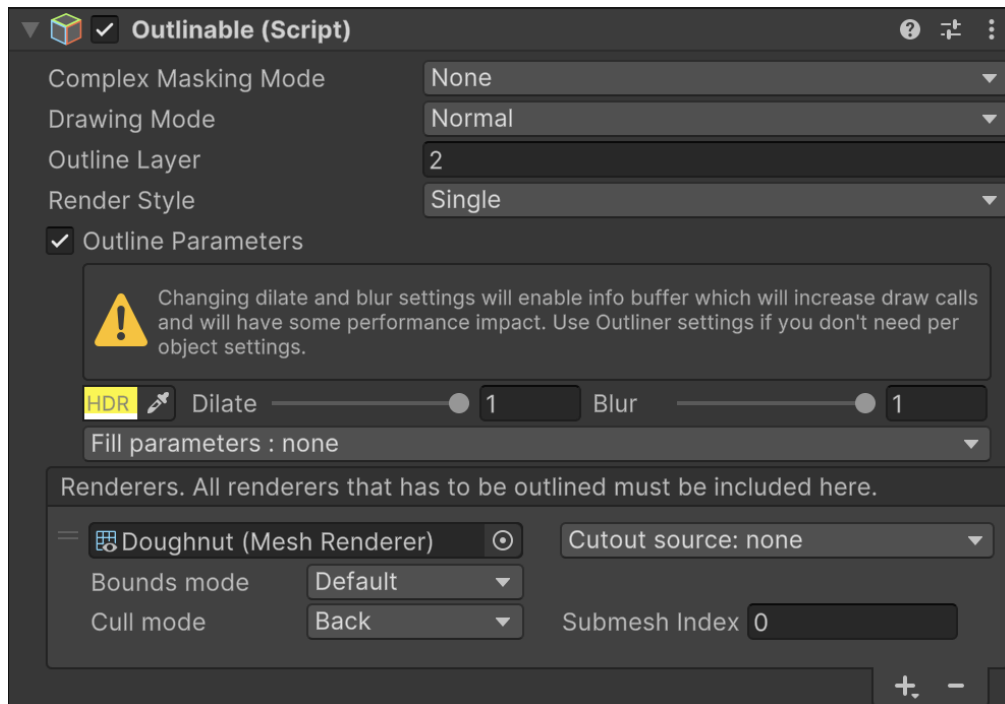


Blur Type

Determines which blur type to apply. The most performant option is **Box Blur**, while the most visually appealing is **Gaussian 13x13**.



Outlinable settings



The **Outlinable** component determines which objects will be outlined by the outline system.

Complex Masking Mode:

- **None**
No masking is applied.
 - **Obstacle Mode**
The outline is visible only behind objects marked as obstacles.
 - **Masking Mode**
The outline is hidden behind objects marked as masks.
-

Drawing Mode:

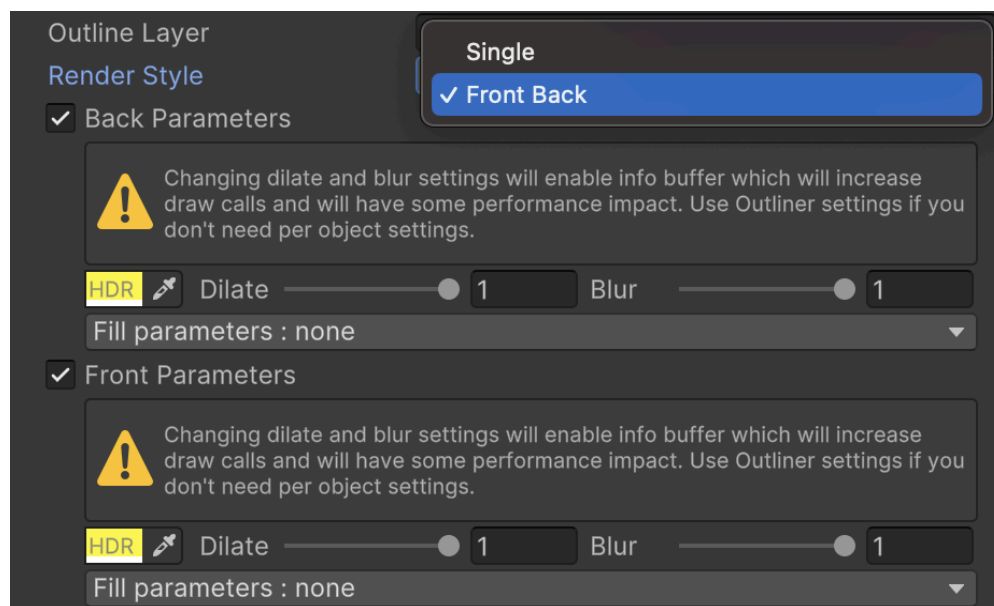
- **Normal**
Draws the outline in the default way.
- **Z Only**
Draws only to the Z buffer. This is useful for objects that usually don't draw to the Z buffer, such as sprites or transparent objects.
- **Generic Mask**
Draws the model with a mask where the outline will not be visible at all.
- **Mask and Obstacle**
Please refer to the **Masking** section for more information.

Outline Layer

Defines the layer of the outline. If you have separate outlines that may overlap, you can assign different layers to the outlinables and add multiple Outliners. In this case, the outlines will not interfere with each other.

Render Style

Determines whether you want to draw only a generic outline or separate it into an obscured or non-obscured outline/fill.



Back Parameters

Defines the parameters that apply to the outline when it is obscured by something (for example, behind a wall).

Front Parameters

Defines the parameters for the outline that is directly visible (not obscured by anything).

Outline Color

Sets the color of the outline.

Dilate Slider

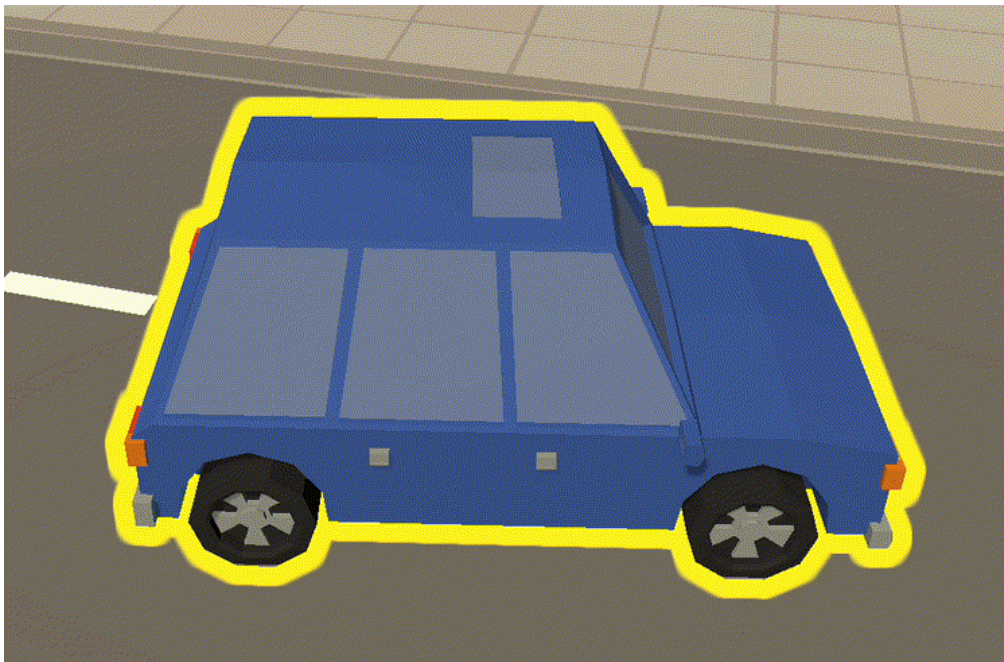
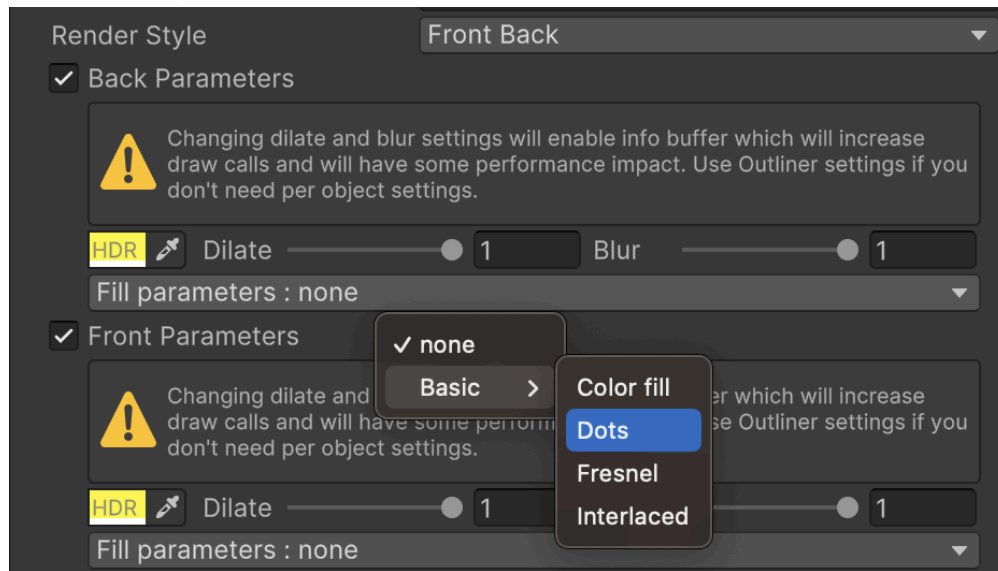
Allows you to change the per-object dilation shift, but it will enable an info buffer.

Blur Slider

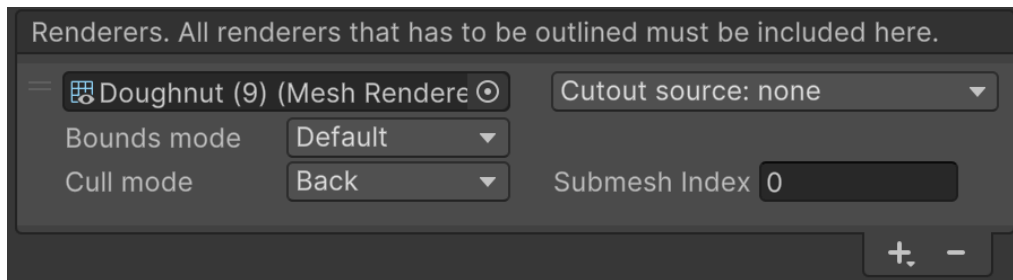
Changes the amount of blur applied to each object, but will also enable an info buffer.

Fill Parameters Selector

A selector for the available fill types. Each type may have different properties, describing how the fill will be rendered for each outlinable. If none is selected, no fill will be applied.



Renderers



- **Renderer**

The renderer that will be used to produce the outline. Typically, this should include all of the renderers associated with your model/object. Disabled renderers will not be rendered.

- **Bounds Mode**

Defines how the renderer's bounds are determined:

- **Default:** Uses the bounds directly from the renderer.
- **Force Recalculate:** Recalculates the bounds every frame for dynamic updates.
- **Manual:** Allows the user to define custom bounds manually.

- **Cull Mode**

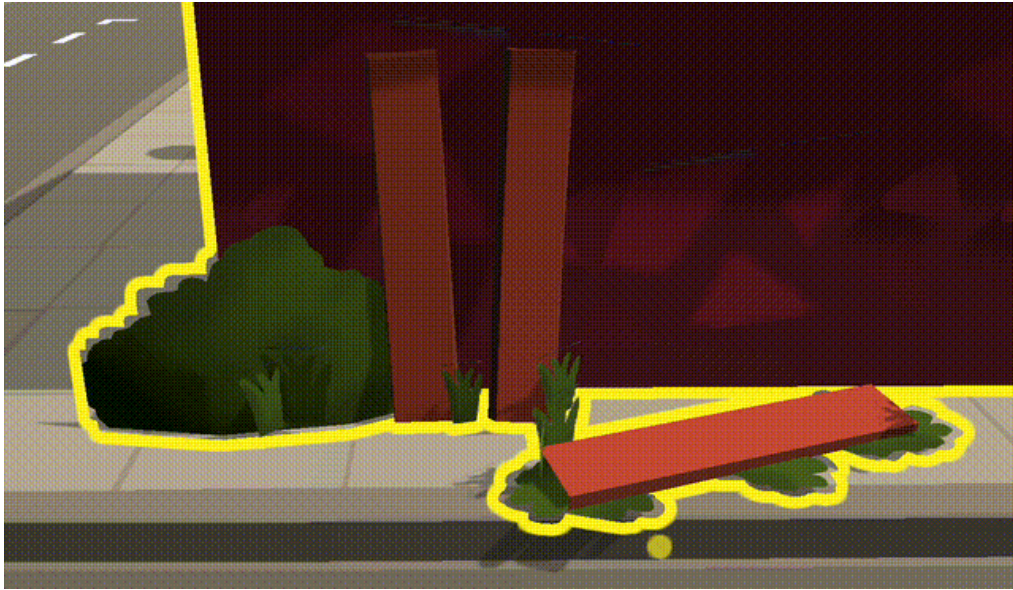
Defines which cull mode should be used to render the outlineable target. Typically, **Back** is the proper choice. If geometry is visible from both sides, select **Off**. The **Front** setting is for extreme cases and is rarely needed, but it is available if required.

- **Cutout Source**

Specifies the texture to be used as the cutout alpha source. When a texture is selected, additional controls become available:

- **Cutout Threshold:** A slider to set the minimum alpha value needed for the texture to be rendered. Pixels with alpha values below this threshold will be discarded.
- **Texture Index:** Useful for texture arrays, this field allows you to specify which texture in the array should be used.
- **Cutout Mask:** Determines which channel of the texture (e.g., R, G, B, or A) will be used for the cutout comparison.

These options provide flexibility for customizing the cutout behavior to fit your requirements.



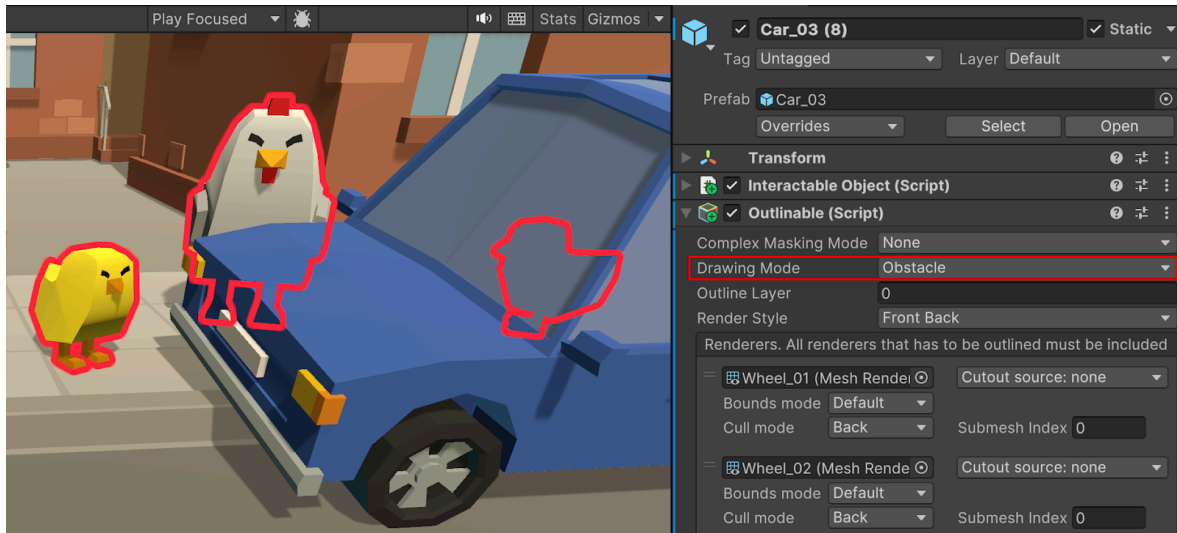
- **Submesh Index**
The index of the submesh used to render the part of the outline.

Advanced Settings

Masking

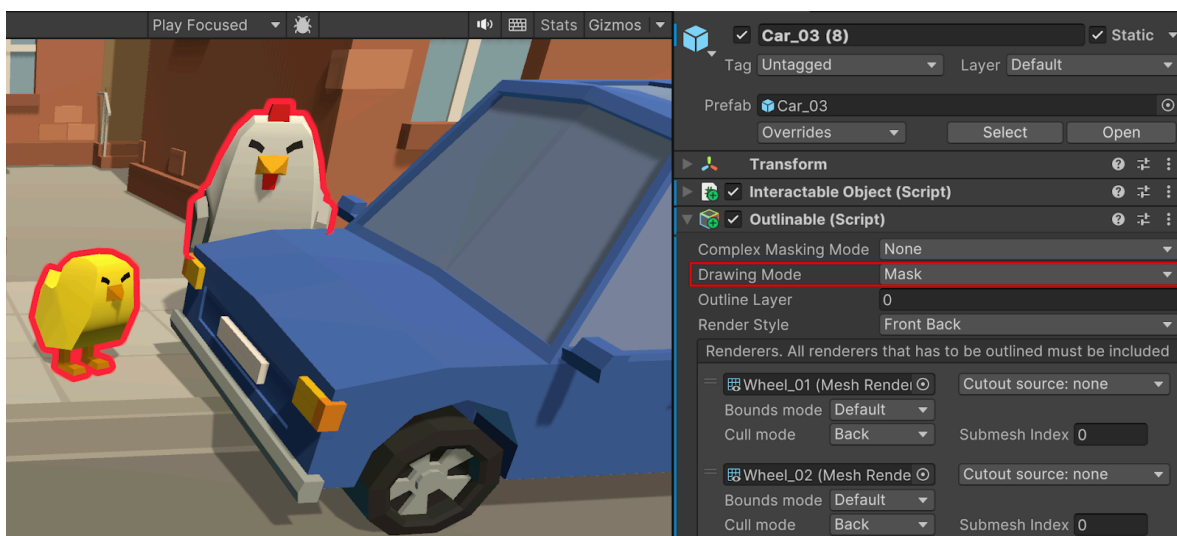
Obstacle Mode

In this mode, the outline is visible only behind objects marked as obstacles. This helps emphasize that the outlined object is positioned behind another, creating a sense of overlap. For example, the characters' outlines are visible only behind the car.



Masking Mode

In this mode, the outline is hidden behind objects marked as masks. This is used to ensure that the outlines are not visible behind certain elements in the scene, providing a cleaner and more precise visual effect. For example, the characters' outlines are hidden behind the car.



Each mode allows for different visual effects depending on the requirements of the scene.

2D Functionality

2D sprites and similar elements typically do not write to the Z-buffer, which is essential for the front-back rendering mode to function correctly. To address this, you can render certain overlapping objects using the Z-only mode, forcing them to write to the Z-buffer.

- **Without Z-buffer writing:**

Objects may not render correctly in front-back mode as their depth is not accounted for.



- **With Z-buffer writing:**

Objects are properly accounted for in the depth buffer, enabling correct rendering and layering in front-back mode.



Integration

Working with the C# API

You can easily interact with the **Outlinable** component via scripting:

1. To access the component assigned to the same GameObject:

```
using EPOOutline;
using UnityEngine;

public class MyScript : MonoBehaviour
{
    private void Start()
    {
        var outlinableToUse = GetComponent<Outlinable>();
    }
}
```

2. To add the **Outlinable** component at runtime:

```
using EPOOutline;
using UnityEngine;

public class MyScript : MonoBehaviour
{
    private void Start()
    {
        var outlinableToUse = gameObject.AddComponent<Outlinable>();
    }
}
```

3. Key properties of the **Outlinable** component:

- Setting render styles:

```
// Render both behind and in front
outlinable.RenderStyle = RenderStyle.FrontBack;

// Render regardless of being obscured
outlinable.RenderStyle = RenderStyle.Single;
```

- Drawing modes:

```
// Normal rendering
outlinable.DrawingMode = OutlinableDrawingMode.Normal;

// Z-buffer only
outlinable.DrawingMode = OutlinableDrawingMode.ZOnly;

// Both
outlinable.DrawingMode = OutlinableDrawingMode.Normal |
OutlinableDrawingMode.ZOnly;
```

- Configuring outline parameters:

```
// Outline color
outlinable.OutlineParameters.Color = Color.green;

// Blur shift
outlinable.OutlineParameters.BlurShift = 0.5f;

// Dilate shift
outlinable.OutlineParameters.DilateShift = 0.5f;

// Disable outline
outlinable.OutlineParameters.Enabled = false;
```

- Managing layers and render targets:

```
// Enable layer 3
outlinable.OutlineLayer = 3;

// Add a renderer
outlinable.OutlineTargets.Add(new OutlineTarget(GetComponent<Renderer>()));
```

- Fill mode settings:

```
outlinable.OutlineParameters.FillPass.Shader = Resources.Load<Shader>("Easy
performant outline/Shaders/Fills/ColorFill");

// Fill color
outlinable.OutlineParameters.FillPass.SetColor("_PublicColor", Color.yellow);
```

Additional examples are available for configuring front and back outline parameters.

DOTween Support

To enable DOTween support, follow these steps:

1. Open the DOTween settings.
 2. Enable outline support directly in the DOTween settings.
-

To use DOTween in your scripts, include the following:

```
using DG.Tweening;
```

You can then easily animate most **Outlinable** parameters. For example:

```
// Animate color
outlinable.FrontParameters.DOColor(new Color(0, 0, 1, 0), 0.5f);

// Animate dilate shift
outlinable.FrontParameters.DODilateShift(1.0f, 0.5f);

// Animate blur shift
outlinable.FrontParameters.DOBlurShift(0.0f, 0.5f);
```

Frequently Asked Questions (FAQs)

Q. Is it easy for beginners to use EP Outline?

A. Yes, there is nothing complicated. After the initial setup, the outliner only requires you to use 2 components.

Q. What are the supported platforms?

A. Android, iOS, PC, Mac, PS4, Oculus/Vive, WebGL. It's probably working with the others but we have not had a chance to check it.

Q. Which versions of Unity are supported?

A. It should work from 2021.3.0. It will probably work on the older versions, but it's not officially supported.

Q. The demo scene is not complete. Where can I find the version I see on the video?

A. We couldn't add a full version due to disagreements with some of the free asset developers. Contact me by email at pirate.parrot.software@gmail.com or reach out on our [Discord](#) server for assistance. Also, please don't forget to quote your invoice number.

Q. Can I use your demo scene in commercial products?

A. Yes, you can. All the models included are free on the asset store.

Q. Does EP Outline support HDRP?

A. Yes. It's supported.

Q. I had some errors after import. Why?

A. Firstly, try restarting Unity and check again. Secondly, try re-importing the asset. If none of these help, please contact me via email at pirate.parrot.software@gmail.com or on our [Discord](#) server.

Q. What is the purpose of the Outliner component?

A. It serves as a general settings provider. You can find more info [here](#).

Q. After adding the outline, my FPS dropped. What should I do?

A. Please check whether the cause of the issue is the outline itself. You can use the profiler to find out what is causing the lag. Try disabling Outliner and check whether it changes the frame rate significantly. Try decreasing the primary buffer resolution or dilate/blur iterations count. It's also important to check it on the target device because the Editor itself has a huge performance impact. If nothing helps, please contact me via email at pirate.parrot.software@gmail.com or on our [Discord](#) server.

Q. After importing the outline, I have pink models. What is happening?

A. This usually happens if you added URP/HDRP support to a project not set up for it. To fix this, remove the URP/HDRP Renderer from the Quality and Graphics settings in Player Settings, or upgrade your materials via **Edit > Render Pipeline > Universal Render Pipeline > Upgrade Project Materials to UniversalRP Materials**. If the issue persists, contact me at pirate.parrot.software@gmail.com or on our [Discord](#) server.

Q. How do I delete the outline plugin without messing the project up?

A. You can remove it from the plugins folder, and everything should be fine.

Q. How do I change the width of the outline?

A. You can play with the dilate iterations count and dilate shift options on the Outliner. You can find more info [here](#).

Contact Information

1. [Asset Store Page](#)
2. Support Email: pirate.parrot.software@gmail.com
3. [Discord](#) Community

Follow Us on Social Media:

- [X \(formerly Twitter\)](#)
- [Facebook](#)
- [Instagram](#)

Links to tutorials and promotional materials:

- [Full Overview of Easy Performant Outline](#) (Installation and Usage)
- [Feature Demonstration of the Asset in Action](#) (Older Video)
- [Configuring Easy Performant Outline through C#](#) (Older Video)

For any questions, feature suggestions, or feedback, feel free to contact us through email or join our Discord community! You can also stay updated by following us on social media.

★ Support Us with a Review! ★

If you found this documentation helpful, please take a moment to leave a review! Your feedback is invaluable in helping us improve [Easy Performant Outline](#) and add new features.

Every review makes a difference — thank you for your support!